COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010- DATA AND STRUCTURE ALGORITHMS	**SECTION:**	CPE21S2
DATE PERFORMED:	08/28/2026	**DATE SUBMITTED:**	09/02/2026
NAME:	MORADILLO, EUNICE E.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

Hands-on Activity 6.1 Searching Techniques

1. PROCEDURE OUTPUT

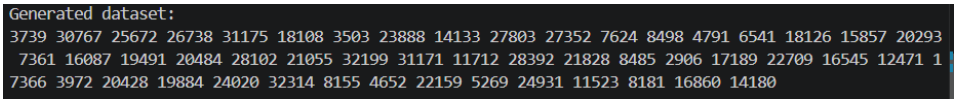
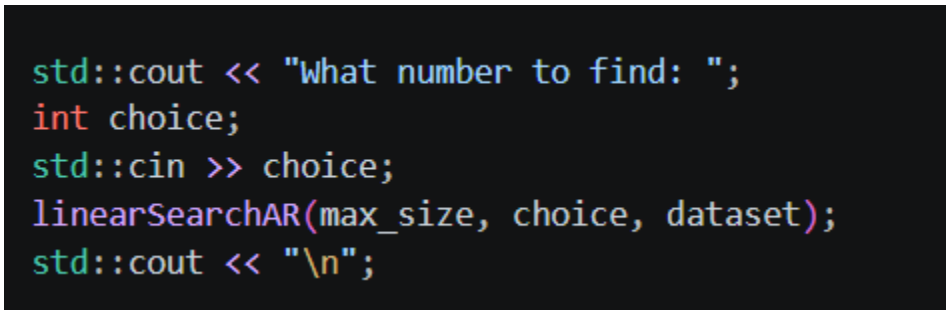
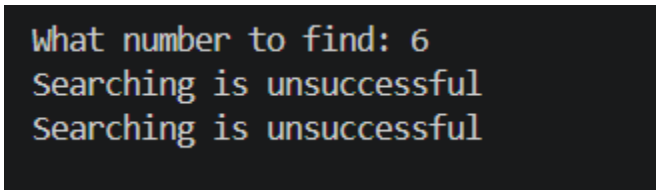
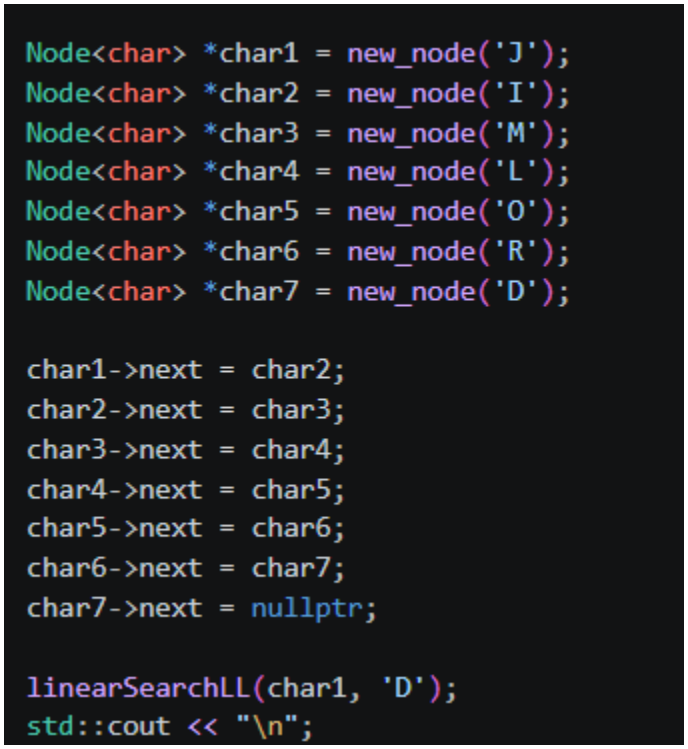
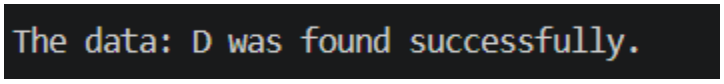
SCREENSHOT	
OBSERVATION	The code generates 50 random numbers each time the program runs. Using the current time as a starting seed makes sure the set of random numbers is different every single time.

Table 6-1. Data Generated and Observations

CODE	
OUTPUT	
OBSERVATION	This code is a search that looks through the array starting from the first item and checks them one by one. If the item is at the very end or nothing, it has to check every single

	number in the list.

Table 6-2a. Linear Search for Arrays

CODE	
OUTPUT	
OBSERVATION	This code is a search that goes through the linked list item by item by following the pointers connecting the nodes. It stops as soon as it finds a matching character or runs out of items.

```
Node<char> *char1 = new_node('J');
Node<char> *char2 = new_node('I');
Node<char> *char3 = new_node('M');
Node<char> *char4 = new_node('L');
Node<char> *char5 = new_node('O');
Node<char> *char6 = new_node('R');
Node<char> *char7 = new_node('D');

char1->next = char2;
char2->next = char3;
char3->next = char4;
char4->next = char5;
char5->next = char6;
char6->next = char7;
char7->next = nullptr;

linearSearchLL(char1, 'D');
std::cout << "\n";
```

```
The data: D was found successfully.
```

Table 6-2b. Linear Search for Linked List

CODE

```
const int arrSize = 10;
int sortedArr[arrSize];
for (int i = 0; i < arrSize; i++) {
    sortedArr[i] = i; // already sorted: 0,1,2,...,9
}

std::cout << "Sorted array: ";
for (int i = 0; i < arrSize; i++) {
    std::cout << sortedArr[i] << " ";
}
std::cout << "\n";

std::cout << "-- Searching for 5 (exists) --\n";
binarySearchArr(sortedArr, arrSize, 5);

std::cout << "\n-- Searching for 100 (does not exist) --\n";
binarySearchArr(sortedArr, arrSize, 100);
std::cout << "\n";
```

OUTPUT

```
Sorted array: 0 1 2 3 4 5 6 7 8 9
-- Searching for 5 (exists) --
Searching is successful

-- Searching for 100 (does not exist) --
Searching is unsuccessful
```

OBSERVATION

This code is a binary search that only works if the list is already sorted from smallest to largest. Instead of checking item by item, it checks the middle element and cuts the list in half each step, making it much faster.

Table 6-3a. Binary Search for Arrays

CODE

```
Node<int> *n1 = new_node(2);
Node<int> *n2 = new_node(4);
Node<int> *n3 = new_node(6);
Node<int> *n4 = new_node(8);
Node<int> *n5 = new_node(10);
Node<int> *n6 = new_node(12);
Node<int> *n7 = new_node(14);

n1->next = n2;
n2->next = n3;
n3->next = n4;
n4->next = n5;
n5->next = n6;
n6->next = n7;
n7->next = nullptr;

std::cout << "-- Searching for 8 (exists) --\n";
binarySearchLL(n1, 8);

std::cout << "\n-- Searching for 20 (does not exist) --\n";
binarySearchLL(n1, 20);

return 0;
```

OUTPUT

```
-- Searching for 8 (exists) --
Searching is successful

-- Searching for 20 (does not exist) --
Searching is unsuccessful
```

OBSERVATION

This code is a linked list that doesn't let you jump straight to the middle item using an index. To find the middle, it uses two pointers, one moving slow and one moving fast, before it can decide which half to search.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM 1

CODE

```cpp
int main() {

    int p1_data[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int p1_size = 10;
    int p1_key = 18;

    // Display the Array list
    std::cout << "Array List: [";
    for (int i = 0; i < p1_size; i++) {
        std::cout << p1_data[i] << (i < p1_size - 1 ? ", " : "");
    }
    std::cout << "]\n";

    int arrComps = linearSearchARCount(p1_size, p1_key, p1_data);
    std::cout << "Array Comparisons needed to find key " << p1_key << ": " << arrComps << "\n";

    // Build and display the Linked List
    Node<int> *head = new_node(p1_data[0]);
    Node<int> *curr = head;
    for (int i = 1; i < p1_size; i++) {
        curr->next = new_node(p1_data[i]);
        curr = curr->next;
    }

    std::cout << "Linked List: ";
    Node<int> *temp = head;
    while (temp != nullptr) {
        std::cout << temp->data << (temp->next != nullptr ? " -> " : "");
        temp = temp->next;
    }
    std::cout << "\n";

    int llComps = linearSearchLLCount(head, p1_key);
    std::cout << "Linked List Comparisons needed to find key " << p1_key << ": " << llComps << "\n";
```

OUTPUT

```
Array List: [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]
Searching is successful (Array)
Array Comparisons needed to find key 18: 2

Linked List: 15 -> 18 -> 2 -> 19 -> 18 -> 0 -> 8 -> 14 -> 19 -> 14
Searching is successful (Linked List)
Linked List Comparisons needed to find key 18: 2
```

z

OBSERVATION

In this code, the Linear search starts at the beginning of a list and checks items one by one. In the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14], the target 18 is in the second spot. This means the program only makes 2 comparisons before finding it. Using an array or a linked list gives the exact same result because both have to be checked item by item.

PROBLEM 2	

CODE	`int k = 18;`
	`int occurrences = countOccurrences(p1_size, k, p1_data);`
	`std::cout << "Count of repeating instances for '" << k << "': " << occurrences << "\n";`

OUTPUT	`Count of repeating instances for '18': 2`
OBSERVATION	This code shows that normal searching stops as soon as it finds a match, but counting matches requires checking the entire list. The updated program looks through all 10 items from start to finish. It keeps track of every time 18 appears, finishing with a total count of 2 times.

PROBLEM 3	

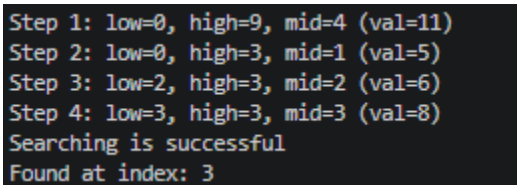
CODE	`int p3_data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};`
	`int p3_size = 10;`
	`int p3_key = 8;`
	
	`int p3_index = binarySearchArrTrace(p3_data, p3_size, p3_key);`
	`std::cout << "Found at index: " << p3_index << "\n";`

OUTPUT	`Iteration 1: low=0, high=9, mid=4 (val=11)`
	`Iteration 2: low=0, high=3, mid=1 (val=5)`
	`Iteration 3: low=2, high=3, mid=2 (val=6)`
	`Iteration 4: low=3, high=3, mid=3 (val=8)`
	`Searching is successful`
	`Found at index: 3`

OBSERVATION	In this code, the binary search works by splitting a sorted list in half over and over. In the sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18], it checks the middle item 11 first. Since 8 is smaller than 11, it throws away the bigger right half and focuses on the left side. It finds 8 at index 3 in 4 step

PROBLEM 4	

CODE	`int p4_index = recursiveBinarySearch(p3_data, 0, p3_size - 1, p3_key);`
	`std::cout << "Found at index: " << p4_index << "\n";`
	
	`return 0;`

OUTPUT	
OBSERVATION	This code is a recursive binary search that uses the exact same logic as Problem 3, but changes how the code runs. Instead of using a loop, the search function calls itself with a smaller section of the list each time. It takes the exact same 4 steps to find 8, proving that loops and recursion give identical results.

CONCLUSION
In this activity, I learned how linear search checks items one by one in arrays or linked lists, while binary search saves time by repeatedly splitting a sorted list in half. In analyzing the procedure, it showed that arrays allow direct index jumps for binary search, whereas linked lists require a slow and fast pointer approach just to find the middle node. For the supplementary activity, linear search found 18 in 2 comparisons for both arrays and linked lists, scanning the full list counted 2 occurrences of 18, and both iterative and recursive binary search located 8 in the exact same 4 steps. I think I did well by successfully writing and running the C++ searching algorithms, though my main area for improvement is getting more comfortable with linked list pointer operations and faster typing skills.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini, copilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code fixing and explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI during online class because I couldn't keep up and my code kept having errors . I also been using bing as google so the ai search or copilot has been helping to explain whenever I use search**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.